ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/351197059

A parallel multipoint viewshed analysis method for urban 3D building scenes

Article *in* Transactions in GIS · April 2021

DOI: 10.1111/tgis.12759

CITATIONS
5

READS
234

2 authors:



Yuxuan Chen
Wuhan University
3 PUBLICATIONS **26** CITATIONS
SEE PROFILE



Jing Chen
Wuhan University
56 PUBLICATIONS **374** CITATIONS
SEE PROFILE

All content following this page was uploaded by Jing Chen on 17 June 2021.

The user has requested enhancement of the downloaded file.

DOI: 10.1111/tgis.12759

RESEARCH ARTICLE

Transactions in GIS WILEY

A parallel multipoint viewshed analysis method for urban 3D building scenes

Yuxuan Chen | Jing Chen

State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing, Wuhan University, Wuhan, China

Correspondence
Jing Chen, State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing, Wuhan University, Wuhan, China.
Email: jchen@whu.edu.cn

Funding information
National Key R&D Program of China, Grant/Award Number: 2017YFB0503703

Abstract

Viewshed analysis is a spatial analysis method essential for three-dimensional (3D) scenes. Existing methods are unable to rapidly and accurately analyze the visibility of complex fields formed by multiple viewpoints in 3D building scenes. In this study, a parallel multipoint viewshed analysis method for urban 3D building scenes is proposed. First, an algorithm was developed for the field-of-view based viewshed scope construction, enabling pickup and calculation of buildings, then a graphical processing unit based 3D scene depth calculation method was used to determine the visibility index for the complex area formed by multiple viewpoints. Results were visualized using a visibility grading method based on the cumulative visibility index. To evaluate accuracy and efficiency, we applied the algorithm to a building scene in an urban area. Results show that the method is able to compute complex viewshed situations formed by multiple viewpoints in urban 3D building scenes.

1 | INTRODUCTION

Viewshed analysis is a spatial analysis method used in geographic information systems (GIS) to characterize the target area observed from single or multiple viewpoints, providing support for the study of spatial cognition (Llobera, 2003). Owing to advances in geographic information and computer technology, there is a growing requirement to deal with visibility issues in three-dimensional (3D) space. Viewshed analysis, based on 3D geographic information technology, not only quantitatively analyzes the visibility of the landscape but also more clearly expresses the 3D spatial structures and geographic information. It is widely used in various fields such as urban environmental assessment and decision-making (Tabrizian et al., 2020), optimal site selection analysis (Hognogi, Pop, Mălăescu, & Nistor, 2020; Petrasova, Harmon, Petras, Tabrizian, & Mitasova, 2018), and targeting of criminals by the police (Vanhorn & Mosurinjohn, 2010).

 © 2021 John Wiley & Sons Ltd

With the development of viewshed analysis, new application requirements have emerged; for example, the needs to deploy the minimum number of cameras to achieve dead-angle-free surveillance, and the minimum number of stations to maximize visual coverage on a given terrain. These requirements can be used to create tourist lookouts (Hognogi et al., 2020), protect landscapes (Petrasova et al., 2018), as well as improve surveillance in firefighting (Cervilla, Tabik, & Romero, 2015), and other uses. To achieve these applications, it is more plausible to consider multiple viewpoints rather than only one viewpoint. Thus, the viewshed analysis based on a single viewpoint, while having a wide range of application scenarios, is not sufficient to support the analysis of the aforementioned needs.

Multipoint viewshed analysis is formed by a series of viewpoints through combining multiple scopes to obtain the maximum range of the viewshed. It is capable of meeting new demands but also emphasizes higher standards on accuracy and efficiency (Yu et al., 2016). The efficiency of the digital elevation model (DEM) based viewshed analysis algorithm can be improved by methods such as optimizing the candidate viewshed regions (Tabik, Zapata, & Romero, 2013) or candidate viewpoints (Wang & Dou, 2020; Yu et al., 2016); however, the multipoint viewshed analysis in 3D scenes is more complicated. Complex 3D scenes in the study area pose a great challenge to the accuracy and efficiency of the analysis, especially in 3D urban scenes, where buildings are densely placed and have irregular facades. Another major difficulty is the ability to evaluate visibility within overlapping areas formed by multiple viewpoints. Owing to these difficulties, there are few studies focusing on analyzing the multipoint viewshed analysis in 3D scenes (Fisher-Gewirtzman, Shashkov, & Doytsher, 2013).

In this article, we investigate how to realize fast and effective multipoint viewshed analysis in 3D building scenes, and propose a parallel multipoint viewshed analysis method for urban 3D building scenes. Unlike traditional methods, this method is based on the shadow map algorithm and performs the calculation and visualization of the multipoint viewshed in the graphical processing unit (GPU) programmable rendering pipeline, which enables the analysis and visualization to run efficiently and finally present intuitive results in the scene. The methods presented in this study include: first, a proposed field-of-view (FOV) based viewshed scope, in which viewshed analysis of a specified scope is achieved by simulating the visual scope of a human, thus avoiding the need for calculation of the entire 3D scene and reducing computational costs; second, a proposed parallel multipoint viewshed analysis algorithm, in which the algorithm achieves all analysis and rendering steps in the GPU, thereby significantly reducing the computation of the 3D buildings in the central processing unit (CPU) and enabling rapid multipoint viewshed analysis; and third, two newly developed visibility indicators, with which the results of the analysis of the target area are visualized in conjunction with the cumulative visibility index.

This article is organized as follows: Section 2 provides a brief description of related works. Section 3 describes the scope of analysis based on the FOV constructed in this study and the proposed parallel multipoint viewshed analysis algorithm for 3D building models. Section 4 discusses the validity and accuracy of the proposed method. Section 5 presents the conclusions and future research.

2 | RELATED WORKS

As an important spatial analysis method in GIS, viewshed analysis has been extensively studied, and several conventional algorithms based on the DEM have been developed (De Floriani, Marzano, & Puppo, 1994; Fisher, 1991, 1993, 1996; Franklin & Ray, 1994; Wang, Robinson, & White, 1996). As increasingly large-scale and higher-resolution terrain data have become available in this century, large-scale DEM-based viewshed analysis algorithms have been well studied. Wu, Pan, Yao, and Luo (2007) proposed a partition-based serial algorithm, which divided the DEM into blocks and adopted the reference plane algorithm to calculate the visibility of the target point on the block. Haverkort, Toma, and Zhuang (2009) designed external memory and used I/O-efficient algorithms to improve performance.

To improve the efficiency of visibility calculations, GPU-based parallel computing technology has been introduced which is capable of performing complex mathematical calculations in GPUs that were previously performed

on CPUs (Dou, Li, & Wang, 2018; Owens et al., 2007; Zhao, Padmanabhan, & Wang, 2013). With the development of GPU programmable technology, the fixed graphics pipeline has transformed into a more flexible programmable pipeline (Owens et al., 2007). This allows the viewshed analysis of actual 3D scenes to be achieved by controlling the rendering process in a programmable rendering pipeline, which is now available for most desktops and laptops. For example, Fang, Yang, Chen, Yao, and Guo (2011) presented a parallel viewshed analysis algorithm based on shadow maps, which uses light and shadow regions to simulate the terrain by creating light sources from the viewpoints. Following this, a real-time parallel viewshed analysis algorithm was proposed by Wang et al. (2015) to create a digital Earth terrain system. It marks the invisible area of Earth by creating an occlusion volume of the target object to delineate visible and invisible areas. Compared to traditional algorithms, these algorithms use the knowledge of computer graphics to achieve real-time calculation in complex 3D scenes, eliminating the dependence on DEMs. In this way, the efficiency of the viewshed analysis algorithms was significantly improved with the parallel computing capability and image processing technology of GPUs. However, the aforementioned studies only analyzed and discussed the case of singular viewpoints, and did not consider multiple viewpoints.

Since multipoint viewshed analysis is computationally expensive as well as time-consuming, certain works have proposed addressing this limitation by zoning the DEM or filtering the candidate points. For example, Tabik et al. (2013) proposed an algorithm for determining the visible points by region and computing the total viewshed on DEMs at a low computational cost, while Cervilla et al. (2015) proposed an algorithm to simplify the combination of multipoint viewsheds, thereby reducing the impact of the number of viewsheds on the efficiency of the algorithm. Yu et al. (2016) proposed a method for filtering candidate viewpoints by partitioning regions, which improved the efficiency of the algorithm by reducing the number of candidate viewpoints. However, these aforementioned improved algorithms are only suitable for DEM-based multipoint viewshed analysis and cannot address the circumstances of complex 3D scenes.

The scene and model data organizations of 3D building scenes are substantially more complex than in DEMs (Fisher-Gewirtzman et al., 2013), which leads to an increased number of factors to be considered when performing multipoint viewshed analysis; for example, the overlapping regions formed by multiple viewpoints and the evaluation of visibility in these regions. Wang et al. (2017) proposed a method of constructing bounding boxes for multiple target points, using the vertices of the boxes for preliminary visibility analysis to filter the target points needing further analysis, thus improving the efficiency of the algorithm. This method is applicable to massive observations and target points, but does not satisfy the complexity of the situation when the target object is an area. Golub, Doytsher, and Fisher-Gewirtzman (2018) designed a 3D visibility model based on the regional bounding box of a target building area, which was used to discuss visibility in the case of multiple viewpoints near the same building within the bounding box. However, that study primarily explained only the visibility of individual viewpoints, with results divided into visible and invisible areas, while lacking consideration of the case where some viewpoints are visible and others are invisible.

Generally, the data organization of 3D building models is more complicated with DEMs. Although existing studies have discussed multiple viewpoints in this scenario, the core operation remains that each viewpoint is analyzed independently, without considering all viewpoints together. Therefore, this study focused on a comprehensive analysis of multiple viewpoints and defined a new indicator in order to visualize the results of multipoint viewshed analysis. The proposed algorithm is based on the GPU programmable rendering pipeline for analyzing a set of multiple viewpoints, and thus avoids extensive computation of vertices, achieving efficient analysis and 3D rendering.

3 | METHODOLOGY

In this study, the multipoint viewshed analysis algorithm is oriented toward 3D building scenes. In this scenario, the applications of viewshed analysis are usually used in human eyes and surveillance cameras as viewpoints, therefore we considered a method for constructing a scope based on the human eyes' FOV, and performed

viewshed analysis for this scope. The proposed parallel multipoint viewshed analysis algorithm for 3D building scenes consists of three stages (see Figure 1). Stage 1 is the data preprocessing stage, stage 2 is the core computing stage, and stage 3 is the cumulative visibility index calculation and rendering stage. Stage 1 is performed in the CPU, while stages 2 and 3 are implemented in the GPU programmable rendering pipeline. Because of the rapid increase in the performance of graphics hardware, coupled with recent improvements in its programmability (Owens et al., 2007), the algorithm is suitable for most desktops and laptops. The three stages of analysis can be described as follows.

Stage 1. Data preprocessing stage: The algorithm constructs the scope by simulating the human eyes' FOV, such that the correspondence between the viewpoint and the analysis scope based on the FOV is first established. At this stage, we determine the multiple viewpoints and their scope, which are used to initially acquire the target buildings. The number of viewpoints is recorded as *N*.

Stage 2. Core computing stage: In this stage, the viewshed analysis in the target FOV is performed by processing the vertex shader and fragment shader in the GPU programmable rendering pipeline. For each viewpoint, the texture sampling points of the building scene in the FOV are first calculated in the vertex shader, and then all sample points are passed to the fragment shader for two depth calculations. By comparing the depth values, it is possible to determine whether the sampling points are visible or not, and a visibility index is then used to record the visibility condition. The visible results of all sampling points within the FOV formed by each viewpoint are then stored in a two-dimensional (2D) texture map. At the end of this stage, *N* texture maps are calculated.

Stage 3. Cumulative visibility index calculation and rendering stage: In this stage, the independent visibility results of *N* texture maps generated in stage 2 are resampled to obtain the visibility indices. Based on the computational power of the GPU, the results are used to calculate the cumulative visibility index in the vertex shader, which is used to grade the viewshed according to the demand. It should be noted that two indicators are set for visibility grading: indicator 1 divides the area into fully visible, partially visible, and invisible areas; while indicator 2 divides the area into visible and fully invisible areas. The grading results are then passed into the fragment shader in the GPU, and the corresponding screen pixel color is set in the shader for visualization.

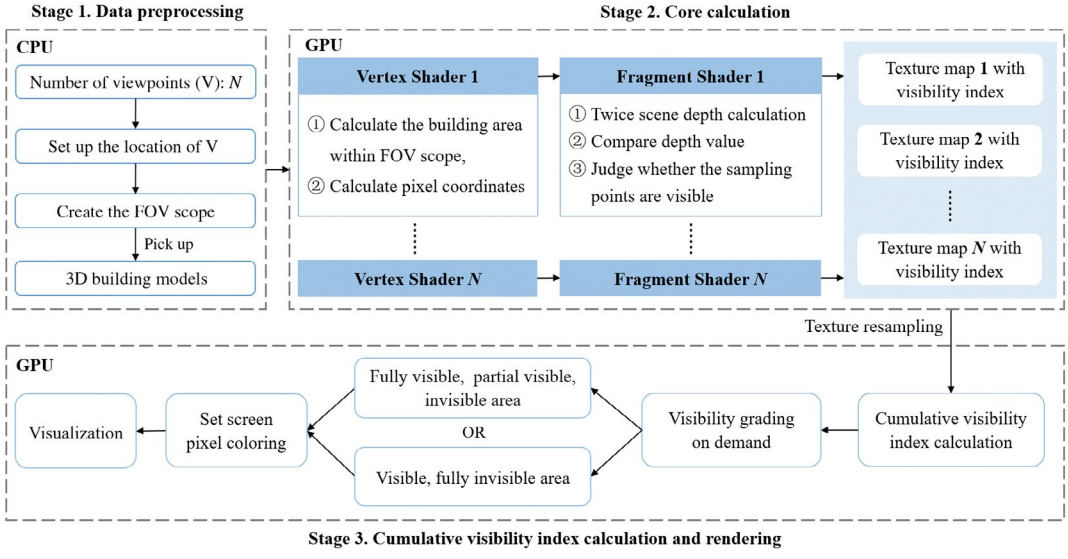


FIGURE 1 Algorithm flowchart. Flowchart of the proposed parallel multipoint viewshed analysis method for 3D urban building scenes

3.1 | FOV-based scope construction for viewshed analysis

Typically, viewshed analysis results are used to distinguish between visible and invisible parts within the viewpoint range. The viewpoint range in GIS is usually defined as a closed area in a 2D plane. Although this area form is relatively simple to construct, it lacks stereoscopic visual effects and selectivity. Therefore, it is mainly suitable for viewshed analysis based on DEM. To explain visibility from the human perspective in a 3D urban environment more intuitively, the concept of the FOV was introduced in this study to construct a 3D scope.

The scope was developed based on Llobera (2003), who proposed using the FOV concept, which considers the observer as the center and the distance and angle of view as the range of the viewshed. In a 3D environment, the construction of the FOV should consider the following three factors: the radian angle of the horizontal line of sight (LOS), the height angle of the vertical LOS, and the farthest distance an observer can see. These factors allow the identification of a stereoscopic region, with the observer's position as the origin, which can be used to simulate the FOV of a single observer. In the observer's FOV shown in Figure 2, the blue region represents the horizontal FOV, while the orange region represents the vertical FOV. By constructing the FOV in this manner, it becomes possible to filter building models in this scope as targets for the analysis, thus significantly narrowing the scope of potential target objects and laying the data foundation for subsequent realization of efficient viewshed analysis. The algorithm for FOV-based scope construction is presented in Algorithm 1. In this algorithm, the FOV-based scope is first constructed in a local coordinate system and then transformed to a world coordinate system using a matrix transformation, which avoids the problem of screen flicker during the scope formation in 3D scenes.

Algorithm 1. FOV-based scope construction algorithm

Input	Viewpoint $O(x, y, z)$ in world coordinates, FOV radius r, height angle $elev$, rad angle rad.
Step 1	Establish a local coordinate system with O as the origin, and calculate the coordinates of the four vertices of the base of the cone in this coordinate system: $A\left(r, \frac{elev}{2}, -\frac{rad}{2}\right), B\left(r, \frac{elev}{2}, \frac{rad}{2}\right), C\left(r, -\frac{elev}{2}, -\frac{rad}{2}\right), D\left(r, -\frac{elev}{2}, \frac{rad}{2}\right)$.
Step 2	Plot the cone FOV V_{local} in the local coordinate system with O, A, B, C, and D.
Step 3	V_{local} in the local coordinate system is transformed into V_{world} in the world coordinate system by matrix calculation as follows: $$V_{world} = \begin{pmatrix} \cos(rad) & -\sin(elev) & 0 & 0 \\ \cos(elev)\sin(rad) & \cos(elev)\cos(rad) & -\sin(elev) & 0 \\ \sin(elev)\cos(rad) & \sin(elev)\cos(rad) & \cos(elev) & 0 \\ x & y & z & 1 \end{pmatrix} \times V_{local} \quad (1)$$
Output	FOV in the world coordinate system V_{world}.

In the case of multiple observers, each located in a different geographic location and target observation range, it becomes necessary not only to construct the FOV for each observer separately but also to consider whether there is an overlap among the multiple ranges. This problem of visibility in overlapping scopes is a challenge in multipoint viewshed analysis. For example, considering the three observers shown in Figure 3. In Figure 3a, the three observers in different positions observed the same target building; however, the three scopes were independent of each other and had no overlap. In Figure 3b, although the three observers were standing in different positions, some overlap of the scopes occurred in the target areas that they observed. Using the overlapping scope of positions A and C as an example, it is assumed that A is visible and C is not. In this case, it cannot be determined directly whether the area is visible or not, and further analysis is required.

As the number of observers increases, the combination of multiple FOV scopes becomes increasingly complex. In this case, there are two solutions. The first solution involves using spatial geometry to calculate all public and non-public areas, and each overlapping scope is analyzed. Then a viewshed analysis and visualization of each area

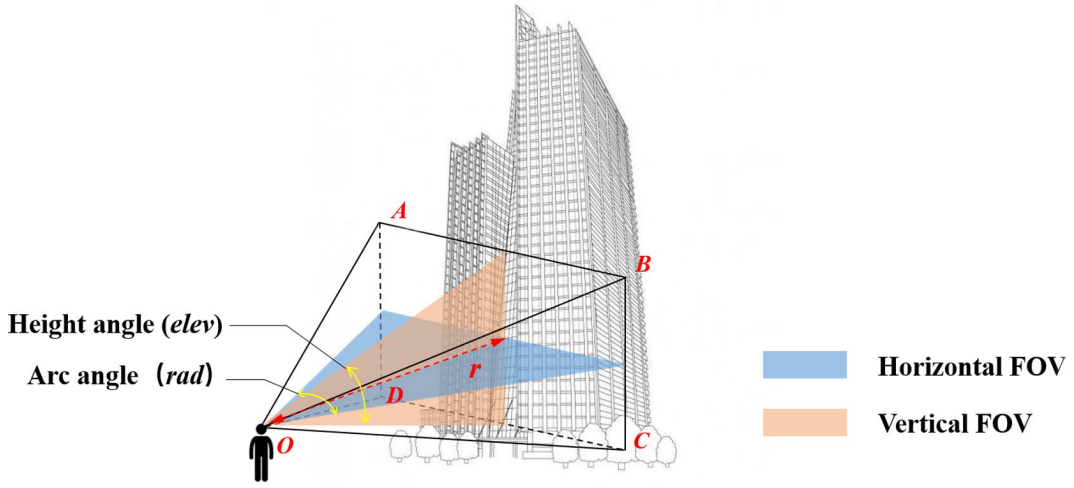


FIGURE 2 FOV-based scope. Construction of an FOV-based scope to enable viewshed analysis of 3D buildings within a specific scope. The FOV is constructed based on three factors: the arc angle of the horizontal LOS (*rad*), height angle of the vertical LOS (*elev*), and the farthest distance an observer can see (*r*)

are performed. The second solution involves an analysis of each FOV-based scope, using computer graphics, in a GPU-programmable rendering pipeline. Then algebraic operations are performed on all the results obtained. Finally, the results are output from the shader and rendered in the scene.

If the first solution is adopted, the geometric difficulties will increase with the number of viewpoints. Therefore, this study adopts the second solution to solve the overlapping scopes problem in multipoint viewshed analysis. This solution significantly reduces the CPU computation cost by introducing parallel computation based on a GPU programmable rendering pipeline.

3.2 | Parallel multipoint viewshed analysis algorithm based on a shadow map

Viewshed analysis in 3D scenes faces two significant challenges. First, intensive computation of the target area in complex scenarios is required, thus reducing the efficiency of the algorithms, a problem which is especially prominent in multipoint viewshed analysis. Second, multi-angle browsing in complex 3D scenes present high requirements for spatial information visualization. Therefore, the visualization of viewshed analysis results should be accurate, intuitive, and reduce the interpretation limitations of the original scene. To solve these problems, the algorithm in this study employs the GPU graphics drawing technique, in which the algorithm is accelerated by GPU parallel computing, and fast visualization is achieved using the GPU programmable rendering pipeline.

The GPU is a specialized microprocessor for graphics rendering with powerful parallel computing capabilities to perform complex mathematical and geometric calculations. Furthermore, the GPU supports complex rendering effects in the graphics rendering pipeline by controlling programmable stages such as vertex and fragment shaders. Based on the GPU programmable rendering pipeline technique, a series of shadow generation algorithms have emerged. Fang et al. (2011), Zhao et al. (2013), and Wang et al.(2015) introduced the principles of these algorithms into their proposed viewshed analysis algorithms. However, these algorithms are limited to the case of a single viewpoint. Therefore, this study introduces a shadow map algorithm based on GPU computation, which focuses on the more complicated case of multiple viewshed formation.

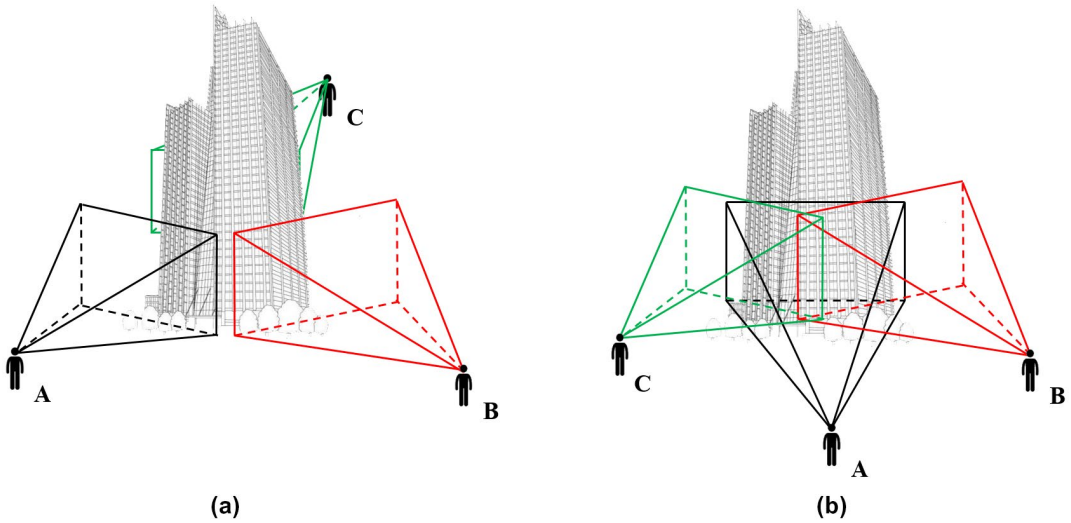


FIGURE 3 Multiple FOV-based scopes. When three observers observe the same building at different locations, the FOV-based scope formed can occur in two ways: (a) indicates that no common area exists among the respective FOV-based scopes of the three individuals, whereas (b) indicates that a common area exists among the FOV-based scopes of the three individuals

3.2.1 | Viewshed analysis algorithm based on the shadow map principle

The shadow map, first introduced by Williams (1978), is a real-time shadow generation technique commonly used in 3D computer graphics. The algorithm implements the shadows in the scene by comparing the depth values obtained from two depth renderings, and consists of two main steps. First, the scene in the light source coordinate system is rendered using the light source as an observer to obtain a depth map that stores depth values. Each pixel value in the depth map represents the depth value, D, of the fragment, closest to the light source, in the scene. Second, the observer is restored to the original camera position to render the scene from that observer and calculate the depth value, Z, of each fragment in the scene from the light source. Then the Z and D values of the same fragment are compared to determine if the fragment is obscured. An outcome of $Z > D$ indicates that the target area is in shadow, while $Z < D$ indicates that the target area is in the light.

In the example shown in Figure 4, an object exists under the light source that forms a shadowed area on the ground. Taking the area P of the object as an example, the distance from the light source to P is D, and the distance from the light source to the ground plane is Z. An outcome of $Z > D$ indicates that the light source is obscured by an object before reaching the target area; therefore, a shadow area is formed on the ground. This region is thus recorded as a shaded area in the depth map.

The principle of the shadow map algorithm is similar to that of the viewshed analysis. Both determine the occlusion of the target area and visualize the two different types of results. The results are represented as light and shadowed areas in the shadow map algorithm, and as visible and invisible areas in the viewshed analysis algorithm. In terms of implementation, the shadow map algorithm is based on GPU graphics drawing techniques, and is computed and rendered, in its entirety, in a GPU programmable rendering pipeline. Therefore, the multipoint viewshed analysis algorithm proposed in this article is based on the shadow map algorithm, in order to better simulate the viewshed analysis in a 3D building scene for a specific FOV scope.

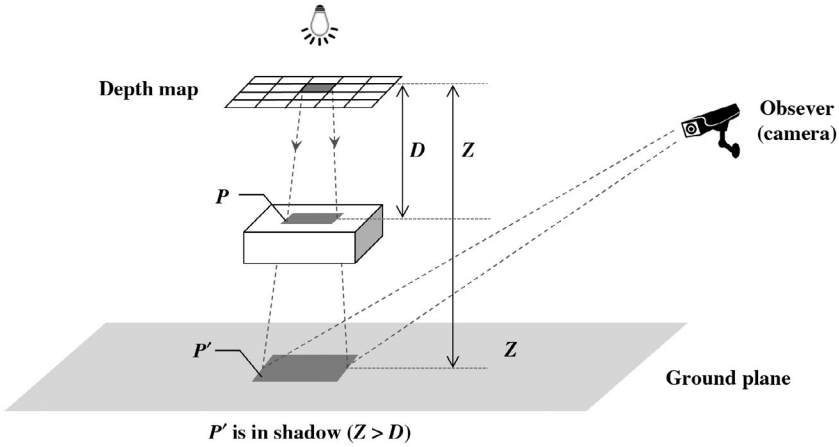


FIGURE 4 Shadow map algorithm. The viewshed algorithm in this study is based on the shadow map algorithm. The shadow map algorithm achieves the shadow in the scene by comparing the depth values *D* and *Z* obtained by two depth renderings. Here, *D* is the depth value of the fragment closest to the light source in the scene, and *Z* is the depth value of the distance of each fragment from the light source in the scene. The *Z* value of fragment is higher than the *D* value of fragment *P*; therefore, the fragment is in shadow

3.2.2 | Depth calculation of 3D scene oriented to the FOV

For the target buildings initially acquired in Section 3.2.1, the viewshed analysis algorithm (Algorithm 2) based on the shadow map is divided into two stages. First, the FOV scope formed by each viewshed is analyzed sequentially in the GPU rendering pipeline to obtain the depth map that stores the depth values of the scene. Second, the visibility index corresponding to the FOV scope is calculated for each viewpoint by texture resampling. Then the cumulative visibility index is calculated and rendered for all resulting visibility indices.

The first stage of the algorithm focuses on building two depth maps of the target scene and comparing the depth values. To achieve the analysis for the specified FOV scope, we considered three parameters before building the depth map: the position of the viewpoint; the direction of the observation; and the target scope of the viewpoint. These parameters were used to set the spotlight point, direction of the light source, and range of the depth map in the algorithm, respectively. The method for calculating the scene depth value on a viewpoint-by-viewpoint basis by combining these three parameters is shown in Algorithm 2.

Algorithm 2. Computation of scene depth for a specific FOV scope

Input	Number of viewpoints N, viewpoint $O_i(x_i,\ y_i,\ z_i)$, center point of the FOV $O_i'(x_i', y_i', z_i')$, radian angle *rad*, FOV in the world coordinate system $V_{world[i]}$ $(i \in [1, N])$.
	While $(i \leq N)$
Step 1	Create 2D textures *depthmap* $[i]$.
Step 2	Create a spotlight source $spotLight_i(x_i,\ y_i,\ z_i)$ at O_i, set the rad of the light source *rad*, and calculate the light direction, $\overrightarrow{lightVec_i} = \overrightarrow{O_iO_i'}$.
Step 3	Use the five vertices in the FOV of the viewpoint $V_{world[i]}$ to construct a cone bounding box to realize the selection of buildings, and add them to the node to be analyzed.
Step 4	For the buildings selected, convert the screen coordinates of the building model vertices $V_{screen[i]}$ to world coordinates $V_{world[i]}$ using $V_{world[i]} = (M_{view[i]} \times M_{proj[i]} \times M_{win[i]})^{-1} \times V_{screen[i]}$ (2) where $M_{view[i]}$ is the model transformation matrix, $M_{proj[i]}$ is the projection transformation matrix, and $M_{win[i]}$ is the viewport transformation matrix.

Step 5	Pass the world coordinates $V_{world[i]}$ to the vertex shader stage in the GPU shading pipeline, and use a ray intersection method to determine whether the point is within the FOV formed by the viewpoint. If the point is within the FOV, it is stored in $V_{inView[i]}$.
Step 6	Apply the following equation in the GPU's vertex shader to calculate the texture coordinate $V_{tex[i]}$ corresponding to the world coordinate $V_{inView[i]}$ of the vertices in the FOV scope: $$V_{tex[i]} = M_{obserview} \times M_{obserproj} \times \begin{matrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 1 \end{matrix} \times V_{inView[i]} \quad (3)$$ where $M_{obserview}$ and $M_{obserproj}$ are the viewpoint matrices and projection matrices in the viewpoint coordinate system. This texture coordinate will be used for the depth comparison. Since the depth map has a texture value range of [0,1], the texture coordinate is shifted and scaled so that each component of the texture coordinate has a value range of [0,1].
Step 7	Pass the texture coordinate $V_{tex[i]}$ into the GPU's fragment shader to generate a depth map $depthmap[i]$ of the scene in the light source coordinate system.
	$i = i + 1$
	End
Output	Depth map depthmap$depthmap[i]$ and $V_{tex[i]} (i \in [1, N])$.

Algorithm 2 can obtain the first depth map generated in the GPU for each viewpoint corresponding to the building scene. Owing to the complexity of the multipoint viewshed analysis, as mentioned earlier, this study adopts a cumulative visibility index calculation for analyzing and comparing depth values to achieve the final viewshed analysis results.

3.2.3 | Calculation of the cumulative visibility index

A single point viewshed analysis results in only two typical cases, that is, the visible and invisible areas. However, because there may be overlapping scopes in multipoint viewshed calculation, the visibility in these areas depends on the analysis results which combine multiple observations. For example, using three viewpoints as an example, the results can be visualized in four ways: none of the three viewpoints are visible; any one of the three viewpoints is visible; any two of the three viewpoints are visible; or all three viewpoints are visible. Therefore, in this study, we modified the visualization criteria for the multipoint viewshed and developed two scenarios based on the actual problem. In the first scenario, the study area is divided into fully visible, partially visible, and completely invisible areas. A fully visible area refers to an area that is visible from all viewpoints; a partially visible area is visible from some viewpoints, and a completely invisible area is invisible from all viewpoints. This scheme is suitable for detailed studies of the visibility of multiple viewpoints, such as the question of whether there is any area that is not visible to all viewpoints. In the second scenario, the study area is divided into visible and completely invisible areas. This scheme is suitable for coarse visible range planning of visual situations formed by multiple viewpoints, such as the problem of maximum visible range. To realize the aforementioned strategy, we introduce the cumulative visibility index (Algorithm 3).

Wheatley (1995) describes the cumulative viewshed as the accumulation of the results of multiple independent viewshed maps, using simple map algebra techniques to produce a new viewshed map. In this section, all independent depth texture map results from Section 3.2.1 were obtained by texture resampling in the GPU shader, and further cumulative visibility index calculations were performed on these results.

Algorithm 3. Algorithm for calculating the cumulative visibility index

Input	Number of viewpoints N, depth map $depthmap[i]$ and $V_{tex[i]}$ $i \in [1, N]$.
	While ($i \leq N$)

Step 1 Access *depthmap* $[i]$ from the GPU's fragment shader.

Step 2 Calculate the depth of the scene *Depth* $[i]$ in the light source coordinate system corresponding to each sample point of the *depthmap* $[i]$:

$$Depth\,[i] = texture2D(depthmap\,[i]\,,\ V_{tex[i]}).\ r \tag{4}$$

Step 3 Calculate the depth of the sample point in normal scene rendering. By accessing $V_{tex[i]}$, a third component (i.e., the *r* component) is retrieved from the texture coordinate of $V_{tex[i]}$, denoted by $V_{tex[i]}.z$. This value is the depth value obtained by the second depth rendering.

Step 4 Compare the depth map information and calculate the visibility index for each viewshed. Determine if the sample point is visible by comparing $V_{tex[i]}.z$ and *Depth* $[i]$. Create a visibility index *visibleIndex* to record the visibility of each sample point (1 if visible and 0 if invisible):

$$visibleIndex[i] = \begin{cases} 0 & V_{tex[i]}.z > Depth[i] \\ 1 & V_{tex[i]}.z \le Depth[i] \end{cases} \tag{5}$$

$i = i + 1$

End

Step 5 Resample all the individual texture maps to obtain the *visibleIndex* for each point. The *visibleIndex* are then summed to calculate the cumulative visibility index *totalVisible*:

$$totalVisible = sum(visibleIndex\,[i]\,)\ i \in [1, N] \tag{6}$$

Step 6 Based on the cumulative visibility index, the visible levels of the area within the FOV are divided, and the screen pixels in the area are set to color according to different level division criteria.(1) Study area is divided into three visible levels. Use the following equation to categorize the completely invisible area as red, the partially visible area as blue, and the completely visible area as green:

$$FragColor.rgba = \begin{cases} (1.0,\ 0.0,\ 0.0,\ 0.7) & totalVisible = 0 \\ (1.0,\ 1.0,\ 0.0,\ 0.7) & 0 < totalVisible < N \\ (0.0,\ 1.0,\ 0.0,\ 0.7) & totalVisible = N \end{cases} \tag{7}$$

The color transparency is set to 0.7 to reduce the shading of the buildings' texture.

(2) The study area is divided into two visible levels. Use the following equation to categorize the completely invisible area as red and the visible area as green.

$$FragColor.rgba = \begin{cases} (1.0,\ 0.0,\ 0.0,\ 0.7) & totalVisible = 0 \\ (0.0,\ 1.0,\ 0.0,\ 0.7) & 0 < totalVisible \le N \end{cases} \tag{8}$$

The color transparency is set to 0.7 to reduce the shading of the buildings' texture.

Output Results of multipoint viewshed analysis

Through the algorithms presented here as well as visualization, multipoint viewshed analysis of 3D building scenes can ultimately be achieved.

4 | IMPLEMENTATION AND ANALYSIS

To verify the feasibility and validity of the algorithm, we performed experiments on multipoint viewshed analysis in 3D building scenes and analyzed the experimental results.

4.1 | Experimental environment and data

The experiment was based on osgEarth, a 3D virtual Earth platform, while C++ and OpenGL shading language (GLSL) were used to implement the algorithm. Note that GLSL is a C/C++ based shading language which can be

executed directly in the GPU programmable rendering pipeline. The study experiments were conducted on a laptop computer equipped with a Windows 10 operating system, Intel Core i7-9750H @ 2.60 GHz CPU, NVIDIA GeForce GTX 1650 GPU, and 8 GB of RAM.

The experiment was conducted in a densely built-up area with an area of 43.46 km^2 in China. Two hundred 3D building models (see Figure 5) were selected as the objects of analysis in the region. Viewpoints 1, 2, and 3 were selected at locations A, B, and C, respectively, to analyze the effect of different viewpoints on the visualization results. Figure 5 shows a 2D vertical view of the experimental area, with individual viewpoints and their horizontal FOVs plotted at locations A, B, and C. It should be noted that a large range of FOVs is shown here, capable of analyzing approximately 20 building models.

4.2 | Experimental results and analysis

The platform osgEarth supports the use of the level-of-detail (LOD) technique to organize and schedule the 3D model. The LOD technique determines the resource allocation for object rendering based on the distance between the model's position and the camera in the 3D scene to reduce the number of faces and the LOD of non-significant objects, thus achieving efficient rendering operations. Essentially, the closer the distance between the camera and the scene, the higher the loaded LOD, and the finer the model. The building models used in these experiments established 13 LODs, with the data structure at each LOD shown in Table 1. In LOD1–LOD7 the building models were not shown because of the long distance between the camera and the scene, and in LOD8–LOD13 the building models with different details were drawn. When the distance between the camera and the scene was within 100 m, the models were in LOD13 (Level 13) and exhibited the finest level of detailed visualization. Therefore, experiments were conducted to check the validity and accuracy of the algorithm at LOD13 (Level 13).

Within the experimental area, the viewpoint locations were first selected, and then the scopes were constructed. For each viewpoint, an FOV (see Figure 6a) was set up as the FOV shed analysis with the rad and height angles of 45°. This FOV enabled the initial detection of building models for subsequent calculations.

Figure 6b–d shows the visualization results when the same building was observed from different angles by a different number of viewpoints. According to indicator 1, the results divided the area into visible, partially visible, and completely invisible areas.



FIGURE 5 Vertical view of the experimental area. A densely built-up area was taken as an example to verify the feasibility of the algorithm in this study. This vertical view shows the three observation points A, B, and C, and their corresponding horizontal FOVs

The case shown in Figure 6b with only one viewpoint had two results: the red invisible area and green visible area. The case shown in Figure 6c with two viewpoints had three results: the invisible area in red, partially visible area in blue, and the fully visible area in green. The partially visible area indicates that the blue area was visible from only one viewpoint. When the number of viewpoints was increased to three (see Figure 6d), the area of analysis formed by the three viewpoints was more complex; however, the results of the viewshed analysis could still be represented as three scenarios based on the division of the area by the cumulative visibility index.

Based on this indicator, it was possible to obtain the area in which all viewpoints were visible, indicating that the target within the area can be focused on from different angles. For example, the activities of criminals could be monitored from different angles to avoid injuries caused by blind spots.

In the practical application of the multipoint viewshed analysis, challenges exist in terms of solving the maximum viewshed, such as determining whether there is a blind spot in the viewing angle among multiple surveillance cameras. While addressing such challenges, subjects are typically more concerned with whether there is at least one viewshed that makes the area visible, and whether there are areas where no viewsheds are visible. Therefore, after completing the calculation of the cumulative visibility index, a second indicator was used to classify the study area into visible and completely invisible areas. In this case, the cumulative visibility index was greater than 0 for the visible area and equal to 0 for the completely invisible area. Using Equation (8) for resetting pixel color and visualizing the results, Figure 7a–c shows the results when the number of viewpoints was 1, 2, and 3 respectively. The visible area expanded with the number of viewpoints, while the blind area of complete invisibility decreased.

The two classification indicators were compared for the same building scenario, as shown in Figure 8. Figure 8a shows the results for indicator 1, while Figure 8b shows the results for indicator 2. Figure 8a shows the existence of areas within the target area that could not be visualized by any viewpoints. Compared to Figure 8a, the visualization of Figure 8b) is more suitable for determining whether there are blind areas that are not visible within the scope constituted by multiple viewpoints.

A comparison of the visualization results of the algorithm proposed in this study and that of Wang et al. (2015) shows that both algorithms use the GPU graphics rendering pipeline with the principle of shading algorithm to analyze the visible situation. Wang et al. (2015) constructed a rectangular area with a single viewpoint as the center and analyzed all the features within the area without further filtering of the objects within the range of analysis. Therefore, that method is more suitable for terrain analysis in larger scenes. For relatively smaller building

TABLE 1 LOD of building model data

Level of detail (LOD)	Distance between the camera and scene (m)	Number of vertices	Number of triangles
13	0–100	321,732	107,199
12	100–300	45,268	8,602
11	300–500	25,812	7,325
10	500–700	12,057	5,945
9	700–1,000	10,365	4,203
8	1,000–1,300	6,458	2,305
7	1,300–1,600	0	0
6	1,600–2,000	0	0
5	2,000–2,500	0	0
4	2,500–3,000	0	0
3	3,000–3,500	0	0
2	3,500–4,000	0	0
1	4,000–4,500	0	0

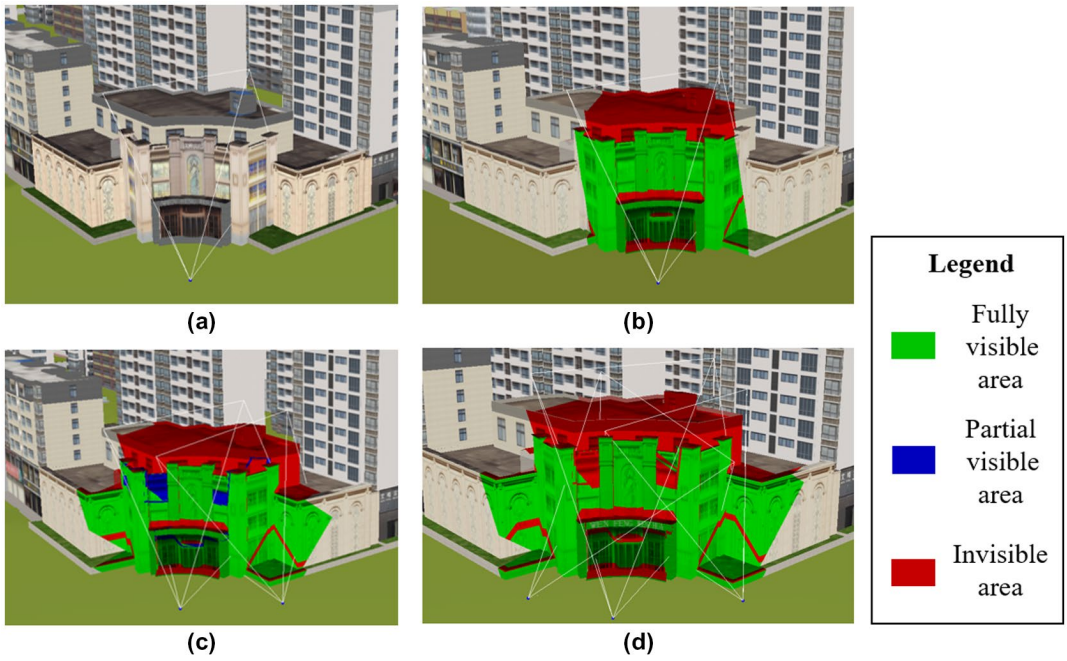


FIGURE 6 Plots of viewshed analysis results for different numbers of viewpoints (indicator 1). The viewshed analysis of the same building was performed using a different number of viewpoints. The visualization method used here divides the study area into fully visible, partially visible, and invisible areas: (a) shows the construction of an FOV-based scope, and the remaining panels show visibility for (b) a single viewpoint, (c) two viewpoints, and (d) three viewpoints

scenes, constructing a flexible visual range to simulate the objects that are realistically observable by the subject is more suitable for practical applications. Moreover, the algorithm presented in this study has significant advantages in multipoint viewshed analysis. Wang et al. (2015) do not consider multipoint viewshed analysis; thus, only one viewpoint can be specified in each analysis. In contrast, the algorithm in this study enables the analysis and visualization of the results by calculating the cumulative visibility index. Therefore, both regions formed by single viewpoints and more sophisticated multiple viewpoints can be visualized.

To verify the efficiency of the algorithm in this study, we further tested the effect of the number of buildings in the analysis range on the algorithm computation time. For this experiment, we completed two analyses of how the number of viewpoints affected efficiency for cases in which, first, the FOV range did not change, and second, the FOV range was broadened, accompanied by a corresponding increase in the number of buildings. The number of viewpoints for this experiment ranged from 1 to 5, and the number of buildings was set to 1, 5, 10, 15, and 20. The total computation time for the entire process, that is, from data preprocessing to the visualization of results, is shown in Table 2.

An analysis of the results in Table 2 reveals that the time required for the algorithm increased gradually as the number of buildings increased. For example, when the number of buildings increased from 1 to 5, the time spent on viewpoints 1 to 5 increased by 0.9326, 1.1035, 1.5810, 1.6079, and 1.8380 times, respectively. By contrast, when the number of buildings increased from 5 to 10, the time increased by only 0.1359, 0.0333, 0.0097, 0.0434, and 0.0742 times for the five viewpoints, respectively. When the number of buildings increased from 10 to 15, the time increased by 0.1740, 0.1554, 0.2024, 0.3308, and 0.3086 times for the five viewpoints, respectively. When the number of buildings increased from 15 to 20, the time increased by 0.1256, 0.1123, 0.1143, 0.0874, and 0.1212 times, respectively.

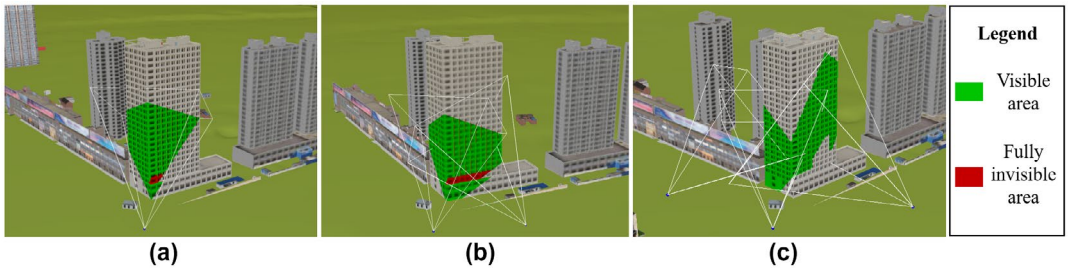


FIGURE 7 Plots of viewshed analysis results for different numbers of viewpoints (indicator 2). The viewshed analysis of the same building was performed using a different number of viewpoints. The visualization method used here divides the study area into visible and fully invisible areas, showing visibility for: (a) a single viewpoint; (b) two viewpoints; and (c) three viewpoints

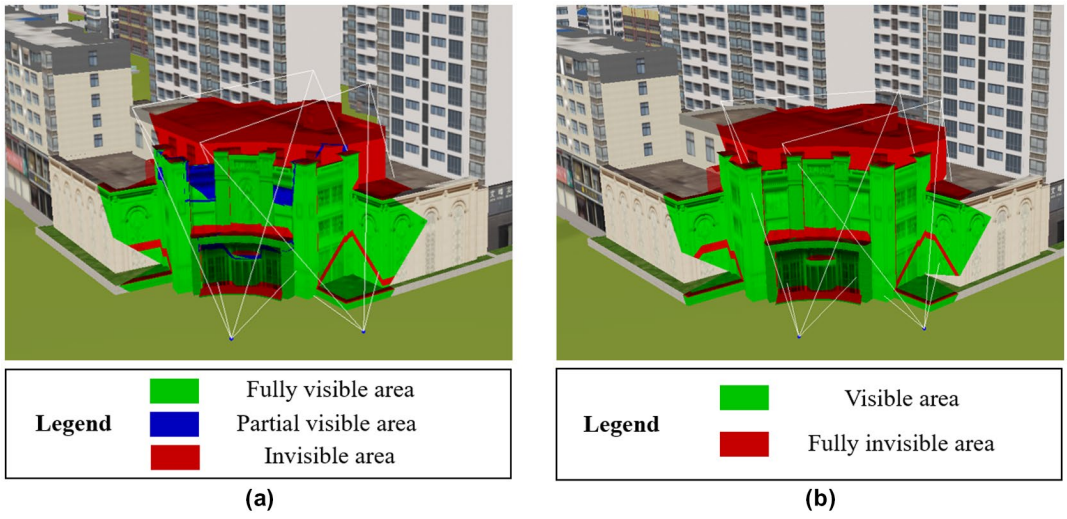


FIGURE 8 Two indicators of the cumulative visibility index. A comparison of the visualization effects of two visible indicators for the same experimental area: (a) shows the division of the area into three levels of visibility, with the blue areas indicating partial visibility, whereas (b) shows the division of the area into two levels of visibility

Overall, the increase in the number of buildings had a limited impact on the speed of the algorithm. This finding can be attributed to the differences between the proposed algorithm in this study and the traditional viewshed algorithm. Moreover, the proposed algorithm does not require extensive interpolation of the target object; thus, an increase in the number of buildings does not cause a significant decrease in the running efficiency of the algorithm.

The results of the effect of the number of viewpoints on the computational efficiency of the algorithm showed an insignificant increase in computation time with an increasing number of viewpoints. Table 3 compares the computational efficiency of algorithms running on CPUs and GPUs. When individual viewpoints were analyzed, the GPU-based algorithm advantage was not distinct. However, as the number of viewpoints increased, the GPU-based algorithm demonstrated the power of parallel computation. Multiple viewpoints could be run in parallel when calculating separate scene depth values, and the computational efficiency was much higher than that of the CPU. The serial computation improved the computing efficiency by an average of 36.20%. Therefore, the algorithm presented in this study has good adaptability in multipoint viewshed analysis.

TABLE 2 Calculation time (seconds) under different number of viewpoints and buildings

Number of viewpoints	Number of buildings				
	1	5	10	15	20
1	0.2375	0.4590	0.5214	0.6121	0.6890
2	0.3226	0.6786	0.7012	0.8102	0.9012
3	0.3232	0.8342	0.8423	1.0128	1.1286
4	0.3356	0.8752	0.9132	1.2153	1.3215
5	0.3321	0.9425	1.0125	1.3250	1.4856

TABLE 3 Comparison of calculation time between CPU and GPU

Number of viewpoints	Calculation time (s)	
	CPU	GPU
1	0.2264	0.2375
2	0.4245	0.3226
3	0.6254	0.3232

Our algorithm still analyzes the multipoint viewshed in static scenes, such as multiple cameras at fixed locations, and first determines the viewpoint and FOV range before completing the subsequent analysis. Therefore, it is not yet possible to analyze dynamic multipoint viewshed analysis. For example, in the case of walking people, viewpoints are moving with the movement, and the FOV ranges are constantly changing with human eyes, which make the viewshed analysis more complicated. In addition, dynamic scenes can generate crowd gathering, and a sharp increase in the number of viewpoints can also lead to a decrease in the efficiency of the algorithm. Thus, our algorithm is mainly applicable to practical applications of camera surveillance.

Finally, the accuracy of the algorithm results was evaluated and compared with the LOS algorithm in the 3D scene (Figure 9). The LOS algorithm, proposed by Doytsher and Shmutter (1994), is conventionally used for determining the visibility between points in a 2D scene. Fisher-Gewirtzman (2014) implemented the algorithm to perform visibility analysis between a viewpoint and a target point in a 3D scene, using green and red line segments to depict the visible and invisible parts, respectively.

In Figure 9, our algorithm and the LOS algorithm were applied to the same viewpoint in the same scene. The result of our algorithm is shown in Figure 9a, while the results of the LOS algorithm are added to the scenario in Figure 9b–d. The yellow points indicate the invisible points of the LOS algorithm, which divided the LOS into two parts: the visible (in green) and invisible (in red). Figure 9b–d shows how the invisible points change with the LOS: the point in Figure 9b is closest to the ground, while points in Figure 9c,d are higher from the ground as the LOS has risen. These invisible points are in the visible area of the building surface, while the area behind them is the invisible area. Thus, the two algorithms show the same visible results, and the accuracy of our algorithm is verified by this experiment.

In this experiment, the LOS algorithm was used to verify the accuracy of our algorithm. Since the LOS algorithm is an algorithm for studying point-to-point visibility, if this algorithm is applied to viewshed analysis, it needs to emit multiple lines of sight from the viewpoint to reach the FOV range, which leads to the low efficiency when performing viewshed analysis. As shown in Table 4, we compared the time used for analysis between the algorithm in this article and the LOS algorithm for 1, 2, and 3 viewpoints, respectively. When there is only one viewpoint, our algorithm took 0.2221 s, while the LOS algorithm took 0.7586s. With the increase of the number of viewpoints, the calculation time of the LOS algorithm increases significantly, while the increasing trend of the time

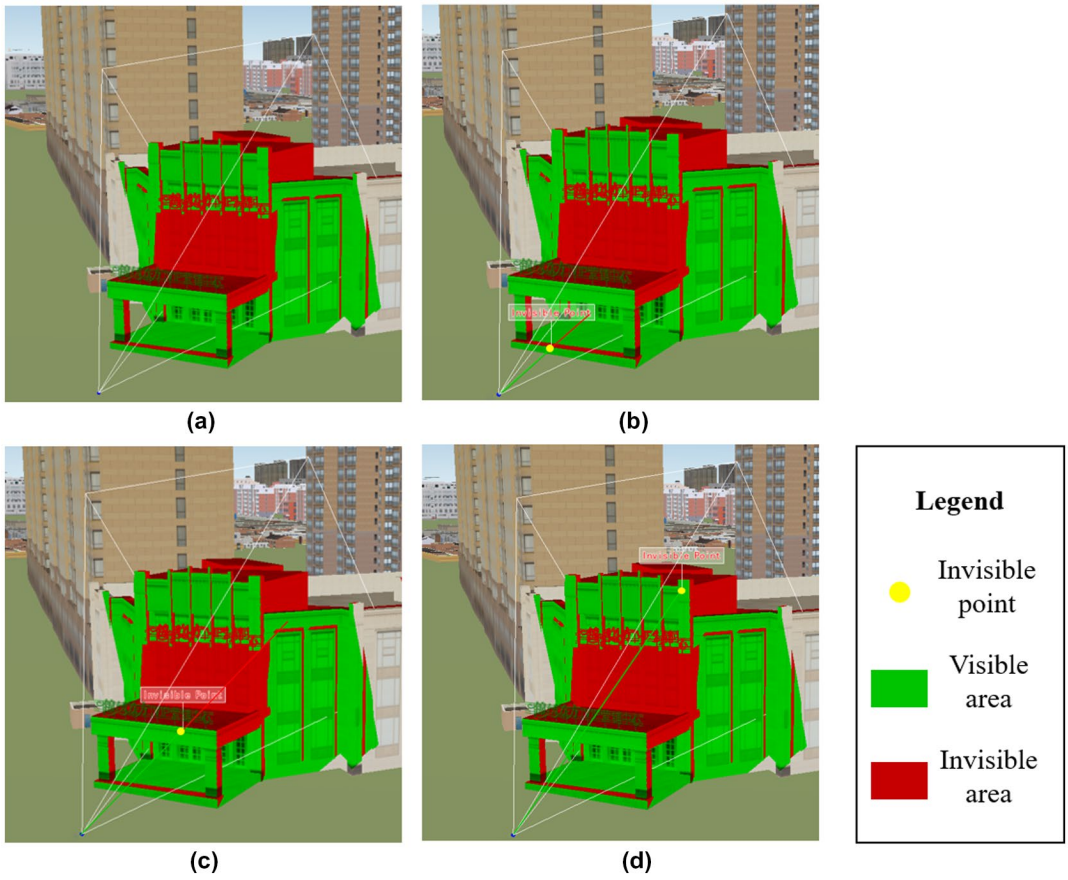


FIGURE 9 Assessment of the accuracy with the LOS algorithm. Our algorithm is compared with the LOS algorithm to evaluate its accuracy: (a) shows the results of our algorithm; and (b)–(d) show the invisible points of the LOS algorithm in the same region. In the figures, the yellow points indicate invisible points, which divide the LOS into a green visible segment and a red invisible segment

TABLE 4 Comparison of calculation time between parallel multipoint viewshed algorithm and LOS algorithm

	Calculation time (s)	
Number of viewpoints	Parallel multipoint viewshed algorithm	LOS algorithm
1	0.2221	0.7586
2	0.3356	1.5234
3	0.3421	2.2646

consumed by our algorithm is more stable. In the case of 1, 2, 3 viewpoints, the time taken by our algorithm is only 29.28, 22.03 and 15.11% of the time taken by the LOS algorithm. This is mainly because an increase of the number of viewpoints leads to a decrease in computational efficiency, thus the LOS algorithm is much less efficient than our proposed algorithm in performing viewshed analysis.

5 | CONCLUSIONS

In 3D building scenes, viewshed analysis usually requires intensive computation of the study target. When the study target is extended from a single viewpoint to a complex area formed by multiple viewpoints, it is essential to focus on analyzing the visible portion of the overlapping scope formed by multiple viewpoints as well as the visible portion of the area corresponding to each viewpoint. Previous studies on viewshed analyses focused on the visible areas formed by a single viewpoint, while studies analyzing and summarizing the viewsheds formed by multiple viewpoints in a 3D scene are few and inadequate.

Therefore, in this study, a parallel multipoint viewshed analysis method for 3D building scenes was proposed. The validity of the method was verified by taking a 3D viewshed analysis of an urban building scene as an example, and the following conclusions were drawn. First, the GPU-based multipoint viewshed analysis algorithm is able to rapidly compute the visible scenario within the complex FOV formed by multiple viewpoints. Compared with CPU-based viewshed computation methods, the algorithm proposed in this study demonstrates the advantage of parallel computing for multipoint processing, with approximately 36.20% improvement in computing efficiency. Second, by constructing an FOV-based scope, redundant computation of invalid scenes is reduced, and simulation of the visual scope in 3D building scenes, such as the human eye and surveillance cameras, becomes possible. This has practical application value. Third, the GPU-based multipoint viewshed analysis algorithm is able to classify the visibility grades of many visualization results analyzed by calculating the cumulative visibility index. This study summarized two different indicators for visualization under different application scenarios, which could be flexibly used to visualize the results according to the specific requirements of the actual problems.

Moreover, our algorithm can compensate for the deficiencies of viewshed analysis based on a single viewpoint in order to more comprehensively consider multiple viewpoints. In addition, the algorithm adopts the GPU parallel computation method, which greatly improves the computational efficiency and is beneficial to the computation in complex 3D scenes. In summary, the algorithm can be applied in the computational and practical applications of viewshed analysis, such as setting up a minimum number of cameras for deadline-free surveillance.

The proposed algorithm is based on rendering texture maps to achieve multipoint viewshed analysis and visualize the results. It shows excellent computational efficiency and visualization effects without exceeding the texture space limit of the texture map. Because the algorithm is currently limited by the texture space, a future focus will be on extending the texture space to accommodate multipoint viewshed analysis for a wider range of scenes. In terms of the target area, the FOV-based scope in this article is a static scope that is better suited for scenarios where multiple cameras are deployed rather than larger crowd subjects, as individuals would have the flexibility to continuously move and change the direction of their observation. Therefore, the effect of dynamic FOV-based scopes on multipoint viewshed analysis should be considered in the future. Moreover, the algorithm proposed in this study is based on a 3D building model. In the future, we will consider extending the supported data formats to include formats such as oblique photography models. This will enhance the decision support provided by multipoint viewshed analysis for urban planning and construction.

ACKNOWLEDGEMENTS

This work was supported by the National Key R&D Program of China (grant number 2017YFB0503703).

CONFLICT OF INTEREST

No potential conflict of interest was reported by the authors.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from the corresponding author upon reasonable request.

ORCID

Jing Chen https://orcid.org/0000-0002-8067-0201

REFERENCES

Cervilla, A. R., Tabik, S., & Romero, L. F. (2015). Siting multiple observers for maximum coverage: An accurate approach. *Procedia Computer Science*, *51*, 356–365. https://doi.org/10.1016/j.procs.2015.05.255

De Floriani, L., Marzano, P., & Puppo, E. (1994). Line-of-sight communication on terrain models. *International Journal of Geographical Information Systems*, *8*(4), 329–342. https://doi.org/10.1080/02693799408902004

Dou, W., Li, Y., & Wang, Y. (2018). A fine-granularity scheduling algorithm for parallel XDraw viewshed analysis. *Earth Science Informatics*, *11*(3), 433–447. https://doi.org/10.1007/s12145-018-0339-5

Doytsher, Y., & Shmutter, B. (1994). Digital elevation model of dead ground. *International Archives of Photogrammetry and Remote Sensing*, *30*, 476–480. https://www.tib.eu/en/search/id/BLCP:CN020351034/Digital-Elevation-Model-of-Dead-Ground?cHash=6d8d81d14cf02ad6013dde8aa4669852

get Fang, C., Yang, C., Chen, Z., Yao, X., & Guo, H. (2011). Parallel algorithm for viewshed analysis on a modern GPU. *International Journal of Digital Earth*, *4*(6), 471–486. https://doi.org/10.1080/17538947.2011.555565

Fisher, P. F. (1991). First experiments in viewshed uncertainty: The accuracy of the viewshed area. *Photogrammetric Engineering and Remote Sensing*, *57*(10), 1321–1327. https://www.asprs.org/wp-content/uploads/pers/1991journal/oct/1991_oct_1321-1327.pdf

Fisher, P. F. (1993). Algorithm and implementation uncertainty in viewshed analysis. *International Journal of Geographical Information Science*, *7*(4), 331–347. https://doi.org/10.1080/02693799308901965

Fisher, P. F. (1996). Extending the applicability of viewsheds in landscape planning. *Photogrammetric Engineering and Remote Sensing*, *62*(11), 1297–1302. http://www.asprs.org/wp-content/uploads/pers/1996journal/nov/1996_nov_1297-1302.pdf

Fisher-Gewirtzman, D. (2014). 3D LOS visibility analysis model: Incorporating quantitative/qualitative aspects in urban environments. In D. Lee, E. Dias, & H. J. Scholten (Eds.), *Geodesign by integrating design and geospatial sciences* (pp. 219–236). Cham, Switzerland: Springer.

Fisher-Gewirtzman, D., Shashkov, A., & Doytsher, Y. (2013). Voxel based volumetric visibility analysis of urban environments. *Survey Review*, *45*(333), 451–461. https://doi.org/10.1179/1752270613Y.0000000059

Franklin, W. R., & Ray, C. (1994). Higher isn't necessarily better: Visibility algorithms and experiments. In T. C. Waugh & R. G. Healey (Eds.), *Advances in GIS research: Sixth International Symposium on Spatial Data Handling* (pp. 751–770). London, UK: Taylor & Francis.

Golub, D., Doytsher, Y., & Fisher-Gewirtzman, D. (2018). 3D visibility analysis indicating quantitative and qualitative aspects of the visible space. *Survey Review*, *50*(359), 134–146. https://doi.org/10.1080/00396265.2016.1253523

Haverkort, H., Toma, L., & Zhuang, Y. (2009). Computing visibility on terrains in external memory. *ACM Journal of Experimental Algorithmics*, *13*, 5. https://doi.org/10.1145/1412228.1412233

get Hognogi, G.-G., Pop, A.-M., Mălăescu, S., & Nistor, M.-M. (2020). Increasing territorial planning activities through viewshed analysis. *Geocarto International*, *36*. https://doi.org/10.1080/10106049.2020.1730450

Llobera, M. (2003). Extending GIS-based visual analysis: The concept of visualscapes. *International Journal of Geographical Information Science*, *17*(1), 25–48. https://doi.org/10.1080/713811741

Owens, J. D., Luebke, D., Govindaraju, N., Harris, M., Krüger, J., Lefohn, A. E., & Purcell, T. J. (2007). A survey of general-purpose computation on graphics hardware. *Computer Graphics Forum*, *26*(1), 80–113. https://doi.org/10.1111/j.1467-8659.2007.01012.x

Petrasova, A., Harmon, B., Petras, V., Tabrizian, P., & Mitasova, H. (2018). *Tangible modeling with open source GIS*. Cham, Switzerland: Springer.

Tabik, S., Zapata, E. L., & Romero, L. F. (2013). Simultaneous computation of total viewshed on large high resolution grids. *International Journal of Geographical Information Science*, *27*(4), 804–814. https://doi.org/10.1080/13658816.2012.677538

Tabrizian, P., Petrasova, A., Korca, P., Vukomanovic, J., Mitasova, H., & Meentemeyer, R. (2020). High resolution viewscape modeling evaluated through immersive virtual environments. *ISPRS International Journal of Geo-Information*, *9*(7), 445. https://doi.org/10.3390/ijgi9070445

Vanhorn, J. E., & Mosurinjohn, N. A. (2010). Urban 3D GIS modeling of terrorism sniper hazards. *Social Science Computer Review*, *28*, 482–496. https://doi.org/10.1177/0894439309360836

Wang, J., Robinson, G. J., & White, K. (1996). A fast solution to local viewshed computation using grid-based digital elevation models. *Photogrammetric Engineering and Remote Sensing*, *62*(10), 1157–1164. https://www.asprs.org/wp-content/uploads/pers/1996journal/oct/1996_oct_1157-1164.pdf

Wang, W., Tang, B., Fan, X., Mao, H., Yang, H., & Zhu, M. (2017). Efficient visibility analysis for massive observers. *Procedia Computer Science*, *111*, 120–128. https://doi.org/10.1016/j.procs.2017.06.018

Wang, Y., & Dou, W. (2020). A fast candidate viewpoints filtering algorithm for multiple viewshed site planning. *International Journal of Geographical Information Science*, *34*(3), 448–463. https://doi.org/10.1080/13658816.2019.1664743

Wang, F., Wang, G., Pan, D., Liu, Y., Yang, L., & Wang, H. (2015). A parallel algorithm for viewshed analysis in three-dimensional Digital Earth. *Computers & Geosciences*, *75*, 57–65. https://doi.org/10.1016/j.cageo.2014.10.012

Wheatley, D. (1995). Cumulative viewshed analysis: A GIS-based method for investigating inter-visibility and its archaeological implication. In G. R. Lock & G. Stancic (Eds.), *Archaeology and geographical information systems: A European perspective* (pp. 171–186). London, UK: Taylor & Francis.

Williams, L. (1978). Casting curved shadows on curved surfaces. *ACM SIGGRAPH Computer Graphics*, *12*, 270–274. https://doi.org/10.1145/965139.807402

Wu, H., Pan, M., Yao, L., & Luo, B. (2007). A partition-based serial algorithm for generating viewshed on massive DEMs. *International Journal of Geographical Information Science*, *21*(9), 955–964. https://doi.org/10.1080/13658810601034218

Yu, T., Xiong, L., Cao, M., Wang, Z., Zhang, Y., & Tang, G. A. (2016). A new algorithm based on region partitioning for filtering candidate viewpoints of a multiple viewshed. *International Journal of Geographical Information Science*, *30*(11), 2171–2187. https://doi.org/10.1080/13658816.2016.1163571

Zhao, Y., Padmanabhan, A., & Wang, S. (2013). A parallel computing approach to viewshed analysis of large terrain data using graphics processing units. *International Journal of Geographical Information Science*, *27*(2), 363–384. https://doi.org/10.1080/13658816.2012.692372

How to cite this article: Chen Y, Chen J. A parallel multipoint viewshed analysis method for urban 3D building scenes. *Transactions in GIS*. 2021;00:1–19. https://doi.org/10.1111/tgis.12759

View publication stats